University of Applied Sciences and Arts Northwestern Switzerland
School of Engineering

# Autonomous Rover Navigation

IP5

Windisch, August 2024



| | |
|---|---|
| **Student** | Miro Albrecht<br>Philippe Krüttli |
| **Supervisor** | Prof. Dr. Christoph Stamm |
| **Client** | Sandro Covo, FHNW Rover Team |
| **Project number** | 24FS_IMVS01 |

University of Applied Sciences and Arts Northwestern Switzerland, School of Engineering

## Abstract

This project presents an improved autonomous navigation system for the rover of the FHNW Rover Team. It is composed of a dynamic pathfinding algorithm combined with a waypoint-following path traversal. A functioning position tracking is a premise of this project and will be implemented in a separate but concurrent project.

The FHNW Rover Team will compete for the third time in a row at the European Rover Challenge (ERC) in September 2024. The ERC is an international competition for universitiy teams from all over the world to develop and challenge their rovers in a Mars-like environment. Various tasks have to be fulfilled, one of them is the navigation task where the autonomous navigation system will be used.

The major improvement in the pathfinding is switching from the A* algorithm to D* Lite for its dynamic approach which allows for a faster and improved path recalculation in case of a detected obstacle or when the rover is no longer on the calculated path. In the path traversal, a behavior tree is used to control the workflow of the rover both in the general navigation from waypoint to waypoint as well as for error detection and handling.

**Keywords:**

Rover, Autonomous Navigation, Pathfinding, Path Traversal, C++, European Rover Challenge

# Contents

List of Figures ..... 1
1 Introduction ..... 2
2 System overview ..... 3
2.1 Node architcture ..... 3
2.1.1 Topics ..... 4
2.2 Simulation environment ..... 5
3 Theory ..... 6
3.1 The navigation task ..... 6
3.2 Pathfinding ..... 6
3.2.1 Cost map ..... 6
3.2.2 Pathfinding algorithm ..... 7
3.3 Path traversal ..... 7
3.3.1 Navigation to a pathpoint ..... 10
3.3.2 Obstacle detection ..... 10
3.3.3 Gazebo simulation ..... 10
4 Implementation ..... 12
4.1 Pathfinding ..... 12
4.1.1 Height map ..... 12
4.1.2 Cost map ..... 13
4.1.3 Pathfinding node ..... 14
4.2 Path traversal ..... 16
4.2.1 Dynamic path recalculation ..... 16
4.2.2 Obstacle detection ..... 16
4.3 Navigation UI ..... 17
4.3.1 Path generation ..... 18
4.4 Testing Days ..... 19
5 Final remarks, conclusion ..... 20
5.1 Pathfinding ..... 20
5.2 Path traversal ..... 20
5.3 User Interface ..... 20
5.4 Outlook and final remarks ..... 20
Bibliography ..... 21
Declaration of honesty ..... 22
Appendix ..... 23
A Original task declaration ..... 24
B Navigation task ERC ..... 25

# List of Figures

Figure 1: ROS topics relevant for the navigation task. ..... 4
Figure 2: Navigation service messages. ..... 4
Figure 3: Illustration of the process from DEM to path. ..... 7
Figure 4: Crab steering visualization ..... 8
Figure 5: Structure of the implemented behavior tree ..... 8
Figure 6: Architecture of the waypoint traversal components in real world ..... 9
Figure 7: Architecture of the waypoint traversal components in simulation ..... 11
Figure 8: Screenshot of the Mars yard model with the height-map-shader applied. ..... 12
Figure 9: Shader used to create a height map from a DEM with Blender ..... 12
Figure 10: Height map of the FHNW campus. Marked with a red oval is a staircase unsuited for autonomous navigation, appearing smooth on the height map. ..... 13
Figure 11: Top-down view of the textured Mars yard model. The blue marks indicate possible waypoint locations for the navigation task. ..... 14
Figure 12: Height map ..... 14
Figure 13: Gradient map ($\sigma = 3$) ..... 14
Figure 14: Traversable map ..... 14
Figure 15: Example for a smooth path.
($\alpha = 1, \text{max_incline} = 30, \sigma = 10$) ..... 15
Figure 16: Example for a rough path.
($\alpha = 45, \text{max_incline} = 30, \sigma = 3$) ..... 15
Figure 17: Simulated rover driving autonomously in Gazebo ..... 16
Figure 18: Simulated rover halting autonomously at waypoint in Gazebo ..... 16
Figure 19: Navigation UI in OpenMCT in idle state ..... 17
Figure 20: Navigation UI in OpenMCT in idle state with heightmap view ..... 17
Figure 21: Waypoint-Creation Dialog in Navigation UI in OpenMCT ..... 18
Figure 22: Waypoints visualization in Navigation UI in OpenMCT ..... 18
Figure 23: Path visualization in Navigation UI in OpenMCT ..... 18
Figure 24: Rover driving autonomously at Testing Day ..... 19

# 1 Introduction

This project contains theory and implementation details to an improved navigation system for the rover of the FHNW Rover Team that will be used at the European Rover Challenge (ERC) in an artificial environment simulating the Mars.

The revised navigation system builds on a previous approach implemented in project *imvs06* in 2022. In the pathfinding stage, an optimal path traversing all waypoints is calculated using the D* Lite algorithm. Waypoints are used to define locations the rover must visit. Once arrived at a waypoint, the rover must stop and wait for a continuation signal from the operator. The path traversal visits every waypoint by following the route and constantly making sure that the rover is on track.

This year's rover *Hufi* has been developed by a team of FHNW students from various disciplines, namely electrical engineering, mechanical engineering and computer science. It is the third rover the team has built with a major redesign from last year's rover following the successful 2nd place at the ERC 2023.

The main use for the autonomous navigation system will be the navigation task at the ERC, however it could also be used for different tasks at the competition. As part of the navigation task, a jury sets 4 different waypoints throughout the Mars yard and the rover has to traverse them, stopping at each of these waypoints such that the judges can measure how precisely the rover navigated. Both time as well as safety concerns have to be accounted for in the pathfinding as well as in the path traversal stages. The path must be efficient yet exclude difficult terrain to prevent the rover from tipping or driving into obstacles placed by the organizers. During path traversal, using higher speeds on long straight paths and lower speeds when closer to a waypoint ensures that both the time limit is kept and the precision is still reasonable.

Each of the rover's components communicate via messages sent over topics in the Robot Operating System (ROS) [1].

A digital elevation model (DEM) is used to create a cost map of the terrain. The cost map is fed into the pathfinding algorithm D* Lite, which calculates the shortest path between two waypoints. Exhaustive search is applied to find the shortest path visiting all waypoints.

A behavior tree is used to control the behavior of the rover during path traversal. The operator can start and resume the autonomous navigation via a user interface.

To provide better quality assurance, a simulation environment is built to test the navigation extensively.

In chapter 2, the surrounding system of the rover, for which the autonomous navigation system has been implemented, is outlined. chapter 3 contains the theory used, followed by chapter 4 with implementation details and test results. chapter 5 concludes with some final remarks, further improvements for future versions and a conclusion on the project.

# 2 System overview

This chapter gives an overview of the hardware and software architecture of the rover. Only the parts relevant for the navigation task are described here.

The rover is controlled using the Robot Operating System ROS. ROS is not an operating system in the classic sense, since it doesn't feature its own kernel. It's a collection of tools and libraries to build applications for robots, using topics and messages for communication between its nodes.

The rover carries a System76 Meerkat [2] mini computer with an Intel i7 CPU and 32 GB RAM. The rover is also transitively dependent on the stereometric camera for the odometry and the motors of the drivetrain to control its movement.

To reduce friction caused by different runtime environments of the development machines and the rover, and because the on-board computer is powerful enough, Docker is used to run ROS. This facilitates the testing and deployment of Docker images with all the required software dependencies pre-installed.

Within the Docker container, a collection of ROS nodes is running. ROS nodes talk to each other by publishing and subscribing to typed messages, called topics in ROS. Request-response-based communication is also possible, which is used by the main navigation node to talk to the path planning node.

A control station is accompanying the rover. They are connected through a local network provided by the rover. Technically, most ROS nodes can be run on both the rover or the control station. But as mandated by the ERC rules, some tasks have to run on the rover itself. The autonomous navigation is such an example. The rover is only allowed to send its (estimated) position to the control station, to help the operator making informed decisions.

## 2.1 Node architcture

The navigation module consists of two main nodes. The first one calculates the shortest path considering the "cost" (incline) of traversing the terrain provided by a previously generated map, from the starting position through the given waypoints including returning back to the start. The cost map is derived from the elevation model, which has to be done once per map and is not done on the rover and therefore doesn't need to run in a ROS node.

The second main node is the navigator. It waits for the waypoints it needs to visit, then requests a path from the path-planning node. After the start signal from the operator, the navigation node traverses all waypoints along the path.

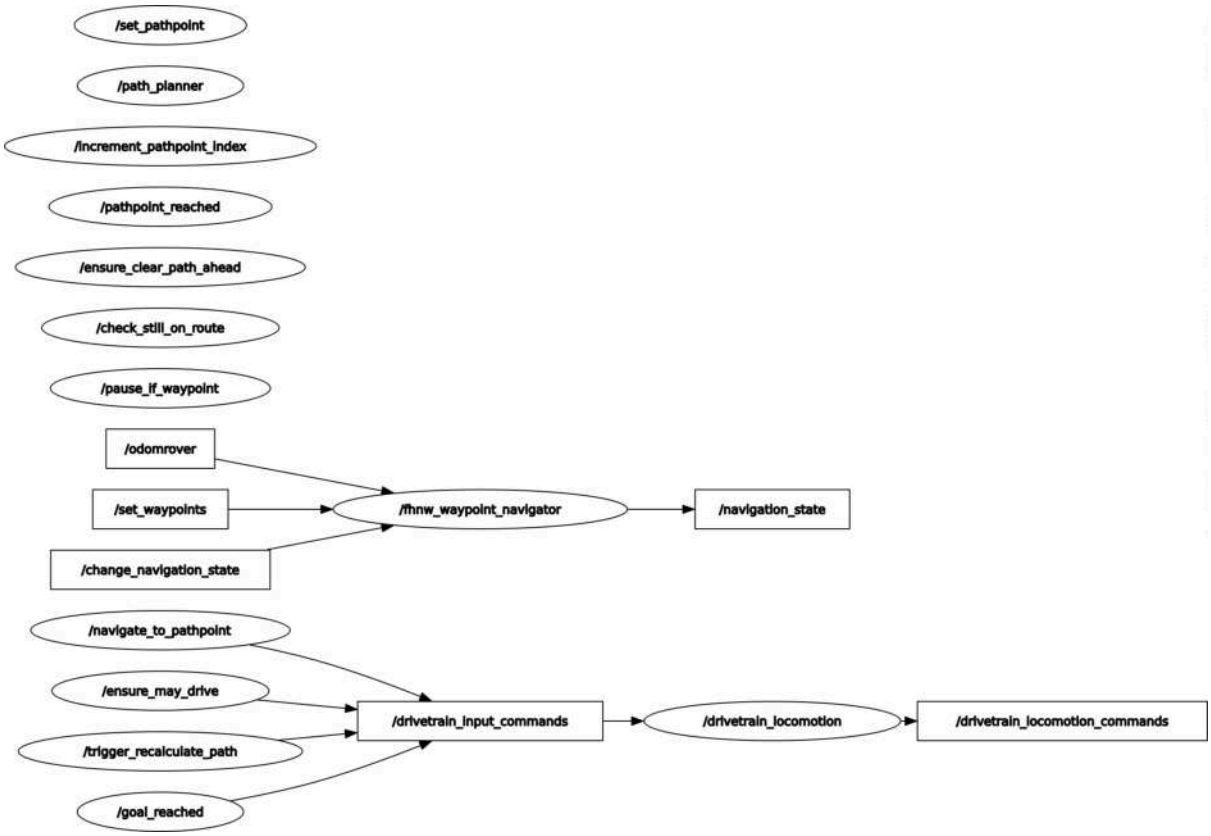



Figure 1: ROS topics relevant for the navigation task.

### 2.1.1 Topics

Figure 1 lists all topics used by the navigation system. Some topics like `/pause_if_waypoint` are used internally. `/set_waypoints` and `/change_navigation_state` allow the operator to control the navigation remotely.

The pathfinding is implemented as a ROS service. The difference between nodes and services is that a service uses sychronous communication, i.e. a request-response model. A service has been chosen for the pathfinding because it better represents the mental model of the relation between the main navigation node and the path planner. When the navigation node needs a path to traverse, it's either blocked by an obstacle or didn't start navigating yet, and therefore can't take any meaningful actions until a path is received. An alternative to services are actions. Similar to services, their communication is request-response based, with the additional ability to report progress until the final result is returned. First, an action was considered for the path planner, but since the pathfinding lacks useful intermediate state to report, the service approach was chosen. The messages the path planner provides are shown in Figure 2.

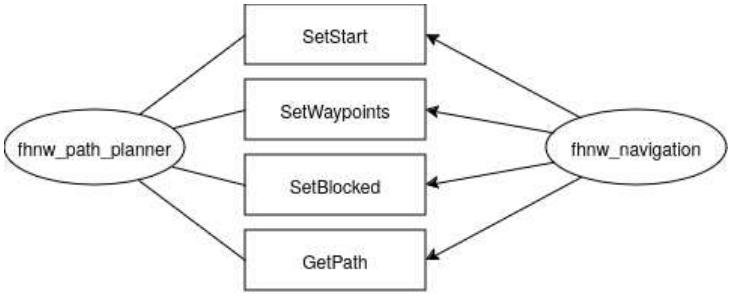



Figure 2: Navigation service messages.

## 2.2 Simulation environment

Gazebo serves as simulation engine and is installed within the Docker environment. It is a widely used software for simulation in ROS/ROS2 projects. This makes it very easy to integrate with existing solutions and use standard ROS topics for communication purposes.

Furthermore, Gazebo contains some free to use models that can be placed in a simulated world to suit the needs of various test cases.

# 3 Theory

This chapter describes the navigation task provided by the ERC organizers as well as the challenges needed to be solved and what improvements upon the previous approach should be taken into account.

## 3.1 The navigation task

One of the tasks at ERC is the navigation task described in Appendix B and in the rulebook published by the organizers at [3]. The goal is to simulate autonomous navigation due to limited bandwidth to a designated location. Along the way, the rover has to reach 4 different waypoints and then return to its starting position. The operator of the rover cannot see any video feedback and the rover must drive fully autonomous in order to receive the maximum points. Using pose information, the team may, however, visualize on a map where the rover is currently located and thereby track its movements. Both the path calculation as well as the processing of data gathered along the way must happen on the on-board computer sitting in the rover itself. The rover has 30 minutes to complete this challenge.

## 3.2 Pathfinding

The pathfinding consists of map creation, path planning and path optimization. Each of these steps can be treated and optimized individually, yet the interface between them, meaning the data shared between components must remain consistent.

### 3.2.1 Cost map

The ERC competitors have access to a digital elevation model (DEM) of the Mars yard where the competition will take place. It is a digital 3D model of the yard, including textures, which is useful for the operator UI. The possible waypoint locations are provided as well.

Theoretically, it's possible to directly use the 3D model for the pathfinding. The obvious disadvantage of this approach is the by magnitudes higher complexity and processing power required to do so. Thus, a much simpler approach was chosen: the first step is to convert the 3D model into a 2D image, with the $x$ and $y$ coordinates representing the corresponding coordinates of the terrain and the value of each pixel corresponding to the height of the terrain at these coordinates. The resulting image is faster to process and it only contains relevant information. This is called a height map (also: depth map, displacement map).

The rover needs to avoid difficult and dangerous terrain. The elevation itself is a useless indicator of the difficulty of the terrain, because it doesn't matter if the rover drives on a low or elevated plane. More interesting is the angle (incline) of the terrain. On flat terrain, the rover can freely navigate. If the terrain is very steep or uneven, the rover could struggle to climb it, get stuck or even fall on the side.

Therefore, the gradient of the height map is taken, following the approach of the previous project [4]. The gradient of the height map contains the incline of the terrain at each pixel's location. To account for small irregularities in the terrain from e.g. small rocks resulting in high incline values, the gradient is smoothed using a gaussian filter. The smoothness is controlled with the parameter $\sigma$.

Each pixel in the gradient map with a value above a certain threshold is marked as untraversable by setting it to a very high (infinitely high) value, forcing the pathfinding algorithm to avoid it. Erosion is applied to the untraversable parts of the map to increase the area for safety. A multiplier affecting the entire map controls the priorization between the shortest path and choosing a longer but safer route.

The following parameters control the process of generating a cost map from a height map:

**`sigma` ($\sigma$)** Sigma of the gaussian filter, controls the extent the gradiant map is smoothed.

**`max_incline`** Threshold for traversable terrain. Values of the gradient map exceeding this limit are considered untraversable.

**`incline_alpha` ($\alpha$)** Controls the tradeoff between short and safe paths. Higher values favor safer paths, lower values favor shorter paths (`incline_alpha` $= 0$ ignores the cost completely).

The result is the final cost map used to find a valid path.

### 3.2.2 Pathfinding algorithm

The previous project used the A* algorithm [5] to find the most cost-efficient path on a cost map. A* is an extension of *Dijkstra's algorithm*, using estimated distances as a heuristic to direct the pathfinding algorithm.

A* is not designed for dynamic routing. D* [6] first sovled this problem, however, it is not commonly implemented due to its complexity. D* Lite [7] is much simpler. Both save their state in a way that only requires to recalculate parts of the path in case an obstacle is detected. Because of its simplicity, D* Lite was chosen to improve the dynamic pathfinding capabilities of the rover.

The pathfinding algorithm only finds the route from one point to another. The ERC rules allow to freely choose the order in which the waypoints are traversed. To find the shortest path visiting all waypoints in no given order is known as the salesman problem. In general there's no efficient solution, but since the number of waypoints is limited to four, an exhaustive search is viable. All permutations are calculated and then the shortest path is returned.

Figure 3 illustrates how a path is obtained starting with the provided DEM.

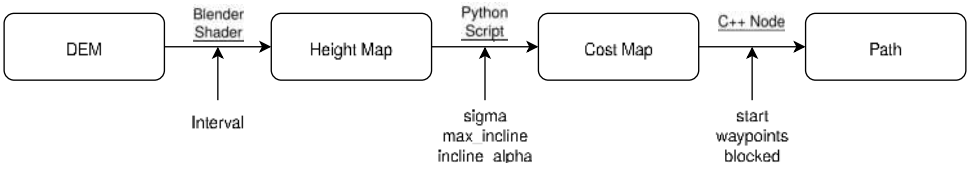



Figure 3: Illustration of the process from DEM to path.

## 3.3 Path traversal

This chapter describes the traversal of the previously calculated path. For path traversal the list of waypoints is needed. That is the list of points the rover must actually reach and where it must come to a stop. The path itself consists of pathpoints where some of them coincide with the waypoints and some of them are points in between two waypoints in order to create a smoother path.

Both pathpoints and waypoints are represented as poses, that is both a three-dimensional position vector as well as a three-dimensional orientation vector. However, for path traversal only the $x$ and $y$ dimensions are considered and therefore navigation happens in 2D space. Considering the third dimension would not make sense, as the rover cannot move up or down. The orientation is meaningless for reaching a waypoint as for now the traversal is implemented only using crab steering. As shown in Figure 5, with crab steering all the wheels are set to the same angle such that the rover can drive a straight line in any direction without having to turn the body itself. This is already an important improvement as mentioned in the thesis [4] describing the first, much simpler implementation of the path traversal.

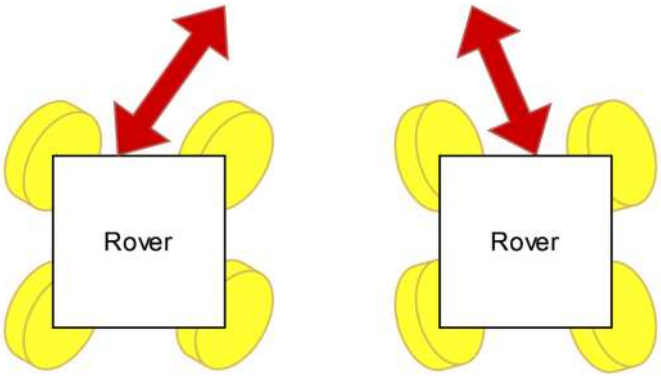



Figure 4: Crab steering visualization

It could further be optimized in a way that would pre-orient the rover such that it would always look towards the direction it is driving but that is out of scope for this project.

Initially, the rover was meant to drive along arclines but for time constraint reasons, as the FHNW Rover Team relies on a working solution by September, the team decided to switch to the simpler navigation using straight lines between pathpoints.

A behavior tree is used to control the behavior of the rover during path traversal as shown in Figure 5. Behavior trees allow for a modular design as every key aspect of controlling behavior is implemented in a separate node of the tree. Later on, the behavior can easily be both adjusted but also extended to suit further needs. In this project, another benefit is that the code is more readable and can be interpreted and adjusted by future team members without the need for multiple days of trying to understand the full codebase.

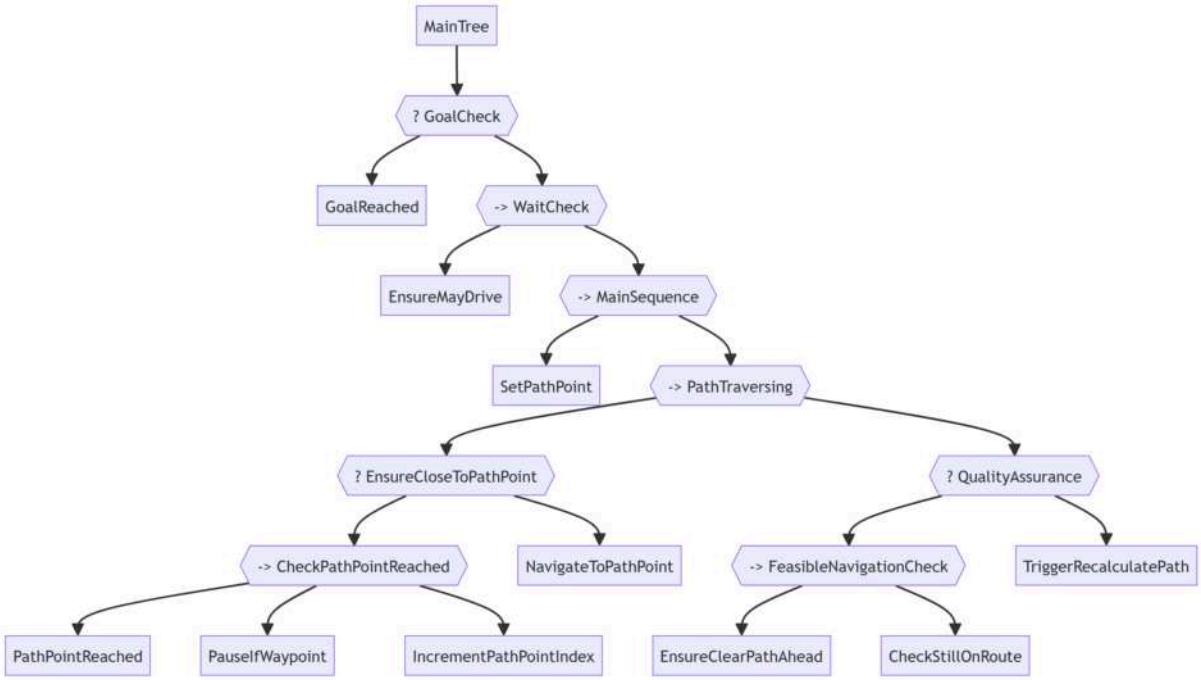



Figure 5: Structure of the implemented behavior tree

The structure of this behavior tree is the current state, yet it can easily be modified in case of changing requirements or newly implemented features.

When the node running the behavior tree is started, it listens to three ROS topics:

- On `/odomrover` the current pose of the rover is being published and the node can therefore determine, whether it is on track or not.
- The Topic `/set_waypoints` is used either manually or by the User Interface to set the waypoints. Upon receiving this message, the node will then call the service `SetWaypoints` which will set the waypoints internally for the pathfinding.
- On the Topic `/change_navigation_state`, the user can either `start`, `resume` or `cancel` the autonomous navigation. Upon a `start` request, the node will call the service `SetStart` to mark the current rover pose as the starting point and then it will trigger the path calculation by calling the service `GetPath`. After the path has been returned, the behavior tree and therefore the autonomous navigation is started. Once a waypoint is reached, the rover stops for the judges to measure the accuracy of the navigation. After that, the team can resume the navigation by sending a `resume` request to the topic `change_navigation_state`. The `cancel` request serves the sole purpose to stop the navigation in case of an emergency of if the team must manually intervene.

Furthermore, it always provides the current navigation state on the topic `/navigation_state` for the UI to display.

All the essential components of the path traversal are listed in Figure 6.

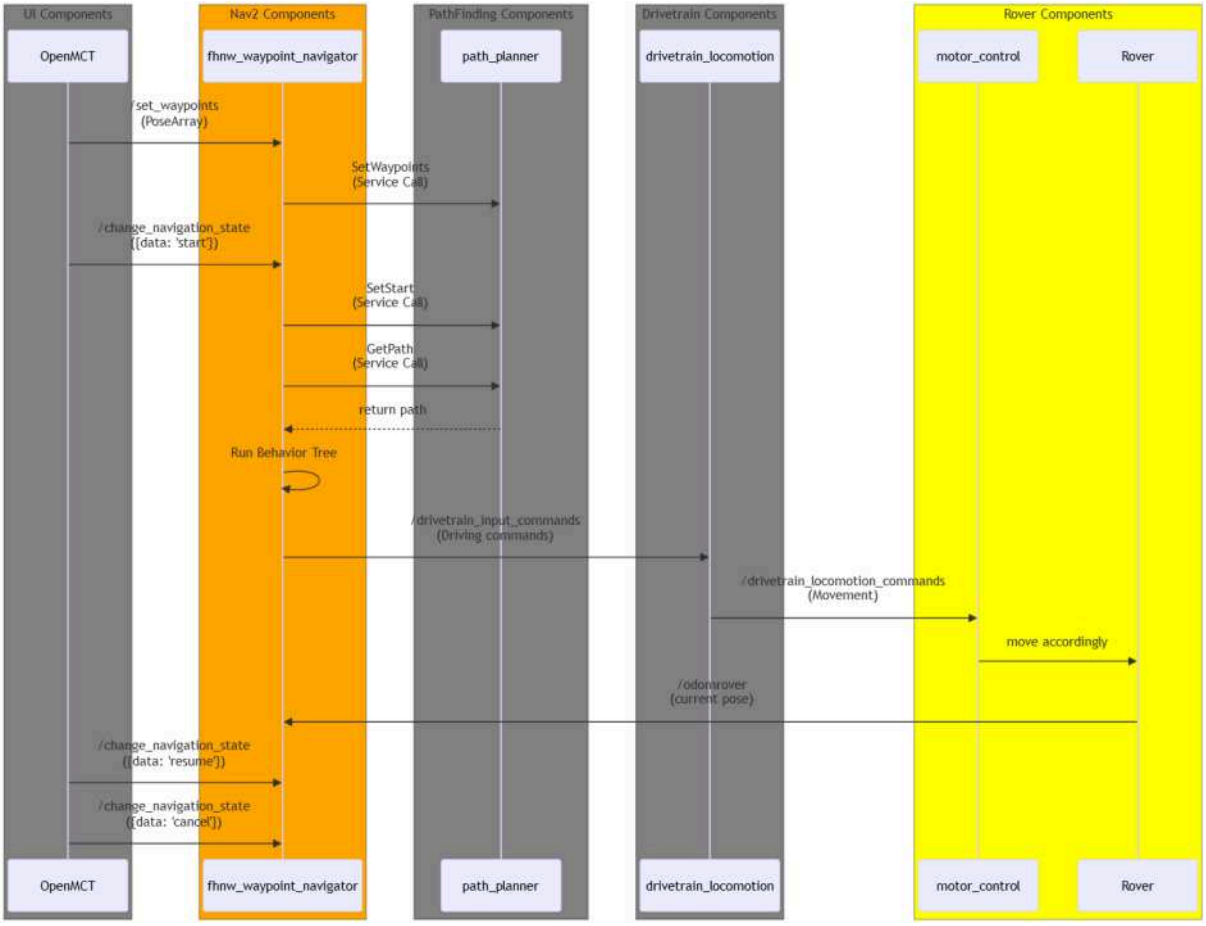



Figure 6: Architecture of the waypoint traversal components in real world

### 3.3.1 Navigation to a pathpoint

Navigation to a pathpoint is implemented by looking at the current position and calculating the straight line towards the next pathpoint. The traversal part may use those straight lines as the path planning already accounts for any obstacles in the terrain and therefore those straight lines do already avoid obstacles and risky terrain.

However, the rover might slip due to weather conditions or other unforeseen situations. To detect that, the node `CheckStillOnRoute` draws the line from the previous pathpoint to the next pathpoint and then checks whether the rover is still on the calculated path within some margin of error.

To check, if point $A$ (in this case the rover's current position) is on a straight line from points $B$ (previous pathpoint) to $C$ (next pathpoint), the following formula is used:

$$\overline{\mathrm{AB}} = B - A$$
$$\overline{\mathrm{BC}} = C - B$$
$$\text{deviation} = \overline{\mathrm{AB}} \times \overline{\mathrm{BC}}$$

If deviation is smaller than some value ε which is the tolerance, then the point is approximately on the line but some boundary checks have to be accounted for as well. Both of the following statements must therefore be true in both $x$- and $y$-axis:

$$A \geq \min(B, C) - \varepsilon$$

$$A \leq \max(B, C) + \varepsilon$$

This ensures that the rover always stays on the calculated path and will therefore never run into any obstacles. However, the path planning must account for the fact that some small deviation is possible and therefore ε left and right as well as above and below the actual path, the terrain must also be clear. Should the rover no longer be on the calculated path, it will recalculate the remaining path dynamically. In contrast to a simple approach which would be to just driving back on the calculated path, this approach ensures that no obstacle is hit as a newly calculated path will account for them.

### 3.3.2 Obstacle detection

In case there is an unforeseen obstacle on the terrain, it has to be detected by the rover. The map provided by the organizers might not include every single obstacle or smoothing of the surfaces when scanning the Mars yard might have led to some obstacles disappearing.

The role of the node `EnsureClearPathAhead` in the behavior tree is simply to detect whether or not an obstacle is ahead and if so, mark it as such using the service `SetBlocked`. Then, the path will be recalculated and thanks to the call to `SetBlocked`, this part of the terrain will not be visited again.

### 3.3.3 Gazebo simulation

In order to test the viability of the autonomous navigation at an early stage, while the physical rover might not yet be finished or out of reach, a simulation is implemented using Gazebo.

The goal of this simulation is to test the behavior of the rover and whether or not the navigation logic is coherent. It is not part of this project to build an extremely realistic simulation with a detailed rover model. Instead, the goal is to mimic the behavior of the rover as closely as possible and make use of Gazebo's provided physics to get a close-enough simulation for first test runs.

However, this simulation does in no means replace physical tests with the rover on-site in Brugg, it just complements them. Figure 7 highlights how only few components (the actual rover components) have to be substituted in order for the navigation to work in the simulation. This is

crucial since the whole navigation stack containing logic remains the same and is therefore actually tested. The only real difference is by what component the movement commands (on topic `/drivetrain_locomotion_commands`) are interpreted and what component returns its pose as odometry data on topic `/odomrover`.

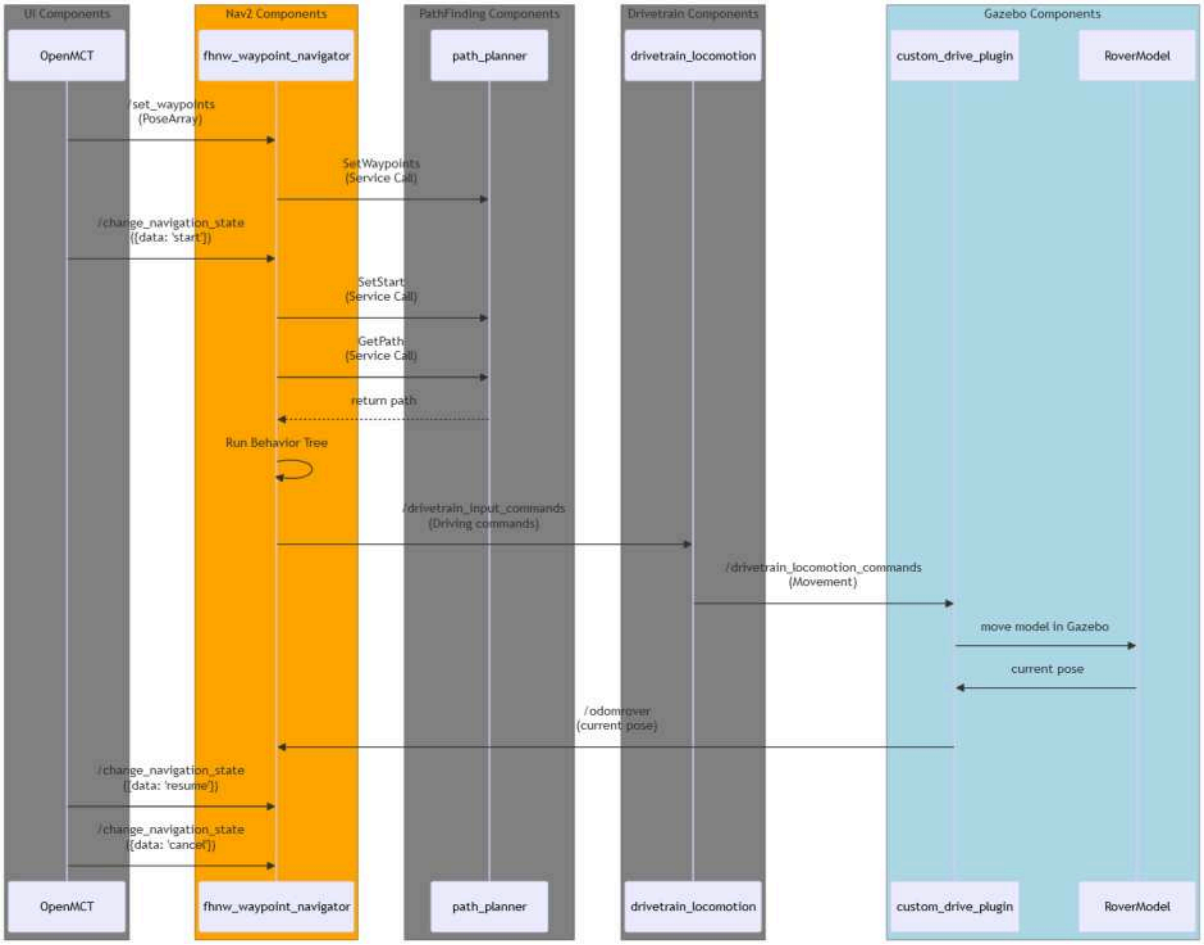



Figure 7: Architecture of the waypoint traversal components in simulation

# 4 Implementation

This chapter outlines the implementation details and results.

## 4.1 Pathfinding

In the previous project, both the cost map generation and the pathfinding algorithm were implemented in Python. In this project, Python was used for the cost map generation due to its mature ecosystem of math processing libraries and low barrier for data visualization with Jupyter notebooks.

The pathfinding algorithm was implemented in C++ according to a decision of the FHNW Rover Team [8]. The main reason for this decision was the Python nodes' lacking stability and reliability.

### 4.1.1 Height map

The cost map is created through a combination of manual work and a Python script. The elevation model (Figure 11) is imported into Blender, a 3D modelling application. A shader is applied, mapping the position in the $z$-dimension to a linear monochrome colorspace (Figure 8, Figure 9). An isometric top-down view of the model is exported as a lossless image (Figure 12 shows the height map of the Mars yard plotted with the Python library `matplotlib`). It's important that the colorspace is linear and not adjusted for human perception. Non-linear colorspaces skew the raw values, creating a distorted view for the script. Before being mapped to color values, the $z$-dimension is clipped to a manually chosen interval. It depends on the DEM if this step is required.

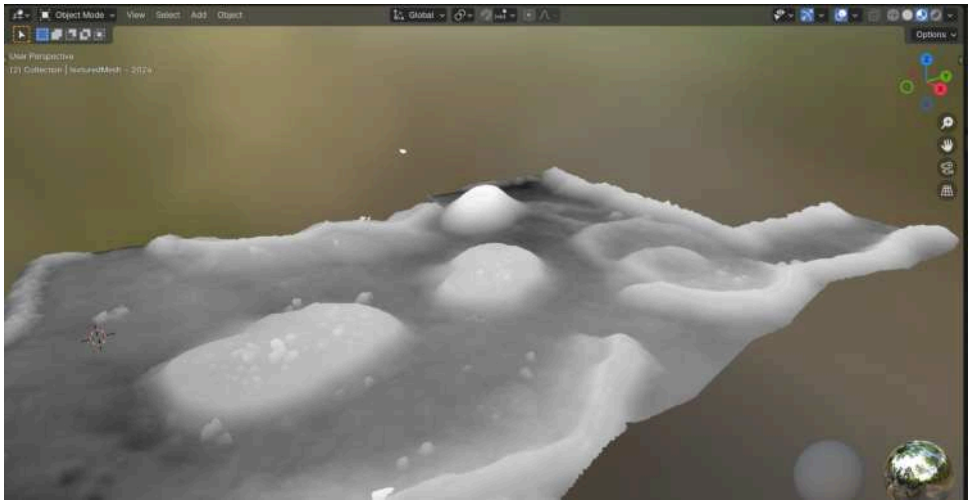


Figure 8: Screenshot of the Mars yard model with the height-map-shader applied.

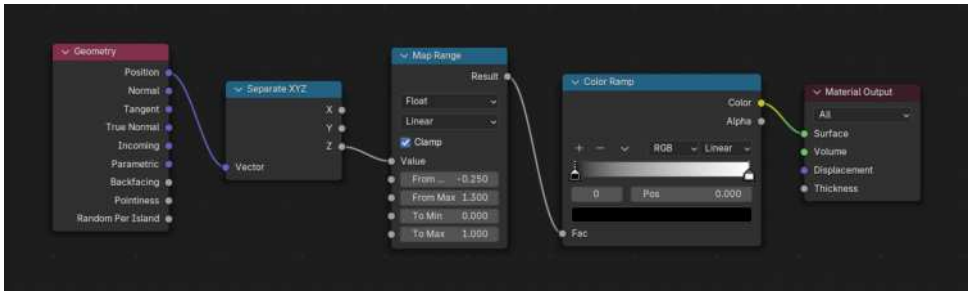



Figure 9: Shader used to create a height map from a DEM with Blender

For example the test DEM of the FHNW campus created with the drone accompanying the rover contained several buildings much taller than the highest location accessible to the rover. Without restricting the range of the $z$-values, the majority of the available resolution is wasted. These parts of the map would be considered untraversable anyway. Additionally, the $z$-resolution of the FHNW campus DEM was already relatively low. This resulted in a height map where the staircase on the campus appeared like a smooth ramp (Figure 10). On the testing day in June, an operator was able to drive the rover up the stairs, however that required careful manual control. The autonomous navigation is not advanced enough for such a maneuver.

For the ERC this issue is not a big concern, since the Mars yard does not contain artificial stairs. The height of the terrain spans over a small range, around $1.5m$ to $2m$. Tall structures are absent, allowing to normalize the $z$-values instead of manually choosing an interval.

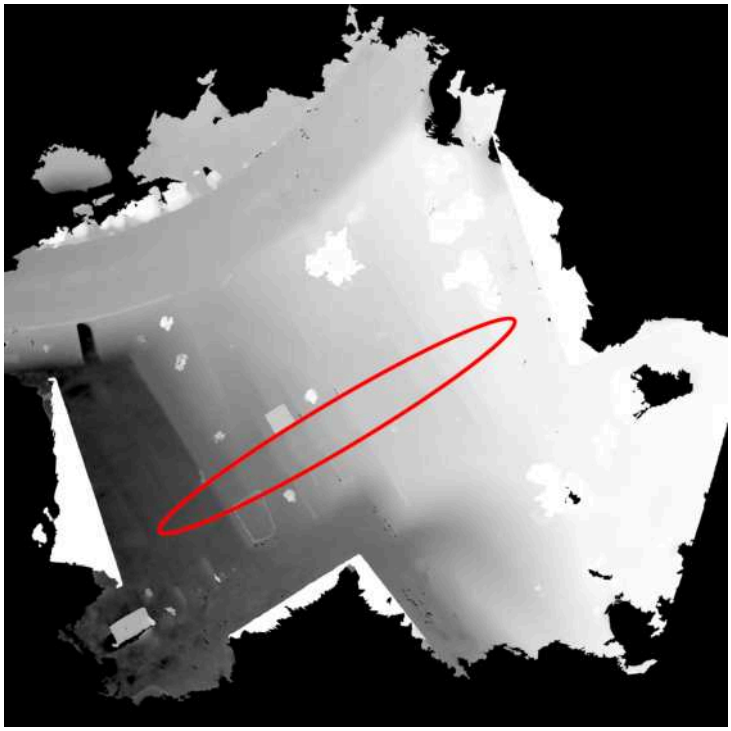


Figure 10: Height map of the FHNW campus. Marked with a red oval is a staircase unsuited for autonomous navigation, appearing smooth on the height map.

#### 4.1.2 Cost map

The resulting height map is then used as the input for the script generating the cost map. The parameters of the script need to be adjusted for each map because they are highly dependent on the resolution of the map and the structure of the terrain. A high resolution map of a rough underground for example could very well be considered untraversable if one has only access to the gradient of that map. The gradient is therefore blurred with a gaussian filter to smooth out small irregularities. Figure 13 shows an example gradient map with $\sigma = 3$ for the gaussian filter. Figure 14 shows an example traversable map.



Figure 11: Top-down view of the textured Mars yard model. The blue marks indicate possible waypoint locations for the navigation task.

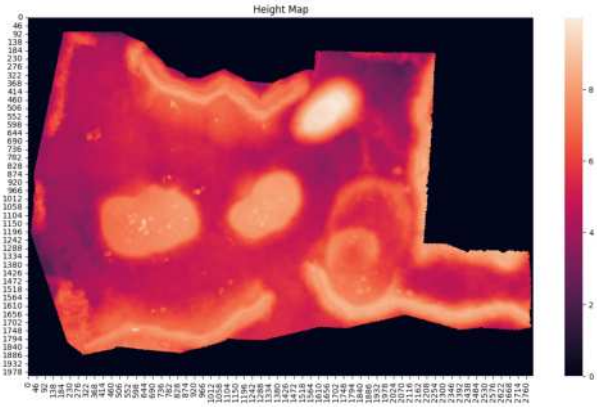



Figure 12: Height map

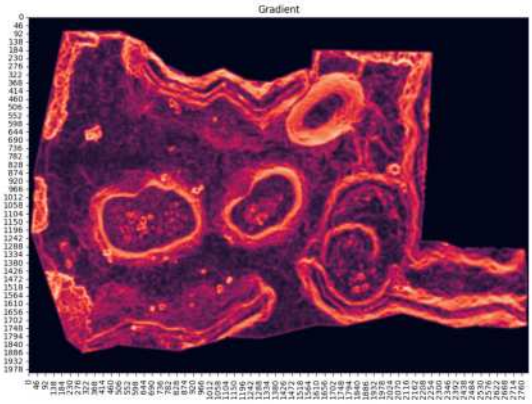



Figure 13: Gradient map ($\sigma = 3$)

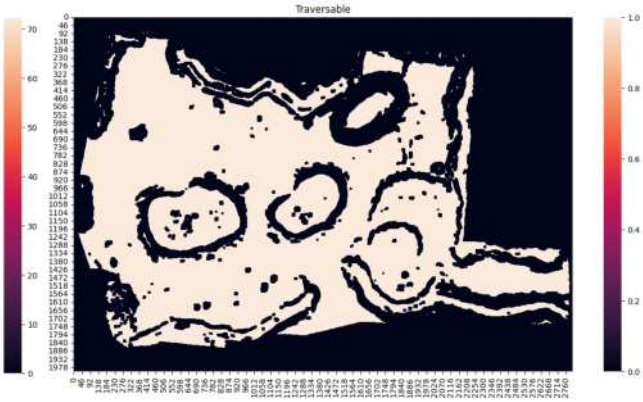



Figure 14: Traversable map

### 4.1.3 Pathfinding node

The endpoints listed below are implemented by the pathfinding node. All parameters are 3D-poses, of the ROS geometry type `geometry_msgs/Pose`. They contain a position as well an orientation. The orientation is currently ignored, same as the $z$-value of the position.

**`SetStart`** Takes a single pose as a parameter, doesn't return anything. A separate endpoint is needed (in addition to `SetWaypoints`) because the path has to start and end at this location and the node therefore has to distinguish it from the other waypoints. Saved for the path generation.

**`SetWaypoints`** Takes a pose array as a parameter, no return value. Saved for the path generation.

**`SetBlocked`** Takes a pose array as a parameter, no return value. Blocked locations are not saved, but instead these positions are directly marked as untraversable on the cost map.

**`GetPath`** This endpoint does not have any parameters. The most cost-efficient path is calculated and returned as a pose array.

`SetStart`, `SetWaypoints` and `SetBlocked` don't have a return value. This could change in the future for better error handling.

When the pathfinding node is started, it loads the cost map from the filesystem. The map is provided as a `.npy` file, a `numpy` array. A library is used to load the data into memory. The node then waits for instructions. The node expects at least one call to both `SetStart` and `SetWaypoints`. For each position a naïve check is performed to confirm the coordinates are within the boundaries of the map. There's no validation for the locations to be on traversable terrain, the check is only in place to prevent out-of-bounds errors. If the check fails, the positions silently discarded.

After the start location and waypoints are set, the navigation node calls `GetPath`. As described above, the path is calculated and then returned. The path can be requested any number of times, with requests to `SetBlocked` freely mixed in. Areas marked as blocked are avoided in all future calls to `GetPath`. Currently there's no way to "unblock" parts of the map, since obstacles are assumed to be static.

Figure 15 features an path example from a cost map with a low `incline_alpha` ($\alpha$) of 1 combined with a $\sigma$ of 10. Compared to Figure 16 the path is a lot smoother, with large parts of the path consiting of almost straight lines. With $\alpha = 45$ and $\sigma = 3$, the path contains many irregularities. Currently, low values for $\alpha$ and high values for $\sigma$ are preferred, since the path is not post-processed. This allows a more streamlined traversal of the path, requiring less turns.

The first test with the rover revealed several mismatches between the coordinate spaces of the odometry, the navigation and the pathfinding. The pathfinding operates entirely in the pixel-space of the cost map, the navigation node however uses meters as a base unit. In pixel-space, the origin $(0, 0)$ is in the top-left corner, the rover's (and therefore the navigation's) origin however is its start position. Furthermore, the $y$-value increases from top to bottom in pixel-space, but the rover's $y$-value increases when driving forward. This issue could be solved in general with a transformation matrix. For simplicity, the pathfinding node only swaps and scales the incoming coordinates as well as applying an offset. This simple approach is possible because when the height map is created, it can be rotated so no non-trivial transformation is required.

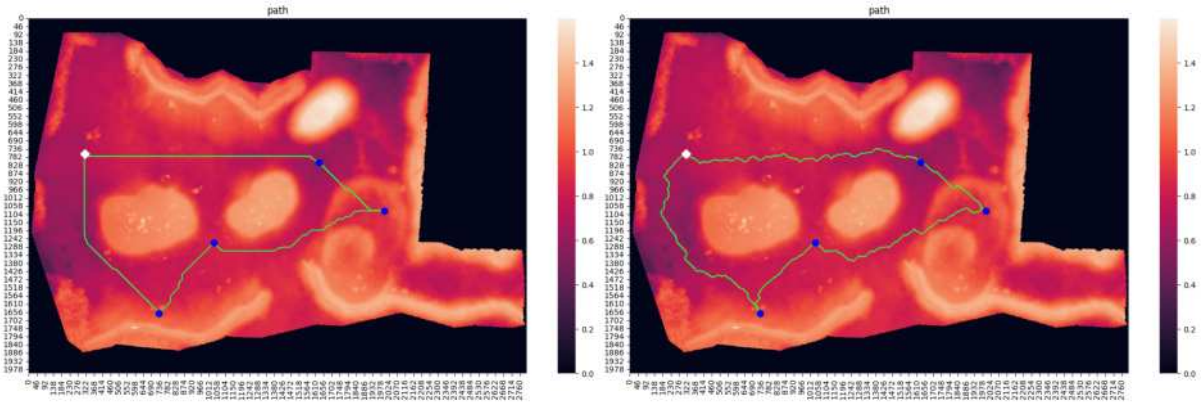



Figure 15: Example for a smooth path.
($\alpha = 1, \text{max_incline} = 30, \sigma = 10$)




Figure 16: Example for a rough path.
($\alpha = 45, \text{max_incline} = 30, \sigma = 3$)

## 4.2 Path traversal

The path traversal has been tested extensively using the simulation. 5 different waypoints have been setup using public models and designated locations such that various driving angles would be needed. The general behavior was as expected, the rover visited each waypoint without issues as shown in Figure 17.

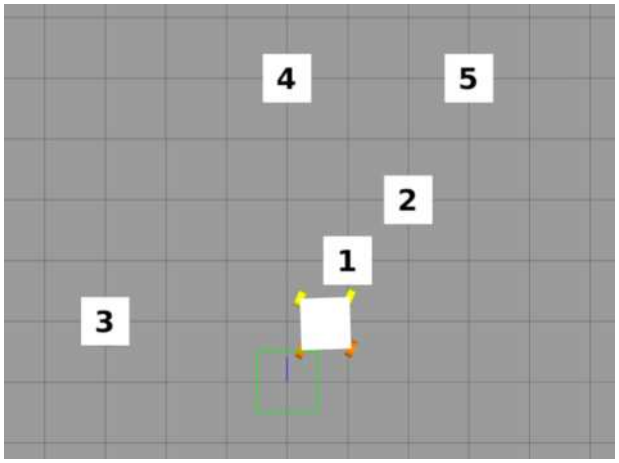



Figure 17: Simulated rover driving autonomously in Gazebo

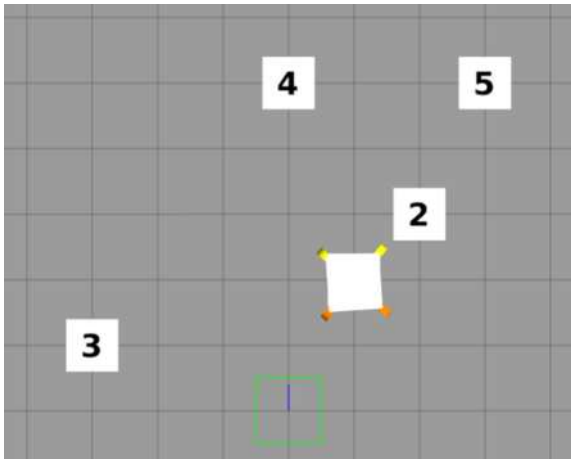



Figure 18: Simulated rover halting autonomously at waypoint in Gazebo

It then waited at each waypoint until the `resume` command was issued. This behavior is shown in Figure 18.

However, in the simulation, the rover turns slightly, yet stays on its path. Since such a behavior could not be observed with the physical rover, is has been neglected in favor of other issues. In future versions, potentially the friction of the rover's wheels might be adjusted in Gazebo to see if this brings any improvement.

### 4.2.1 Dynamic path recalculation

The dynamic recalculation of a path in case of an obstacle or an unforeseen displacement, leading to the rover no longer following the calculated path, is implemented but remains to be tested on the final Testing Day before the ERC. This Testing Day will take place on August 19, 2024.

### 4.2.2 Obstacle detection

Obstacle detection during path traversal has not yet been implemented. The according node in the behavior tree now just always succeeds without taking any action. The goal is to have a working solution for this until ERC in September 2024.

The missing part is the actual detection of an obstacle in the direction the rover is currently driving. This remains to be implemented using the cameras on the rover's tower.

## 4.3 Navigation UI

The user interface is built within the existing UI framework OpenMCT. The FHNW Rover Team had already used OpenMCT in the most recent competition. OpenMCT is a technology developed by NASA providing a base framework with a plugin system to plan and execute space-related missions as described at [9]. It is open source and can be extended with custom plugins.

One such plugin had been developed for the previous competition in order to control the autonomous navigation system. However, as the system has changed in many ways, the UI had to account for these modifications as well.

As shown in Figure 19, the navigation UI shows a picture of the Mars yard and has a green arrow representing the rover's position and orientation. Operator actions can be taken using the buttons below. The buttons are shown (and hidden) based on the current navigation state.

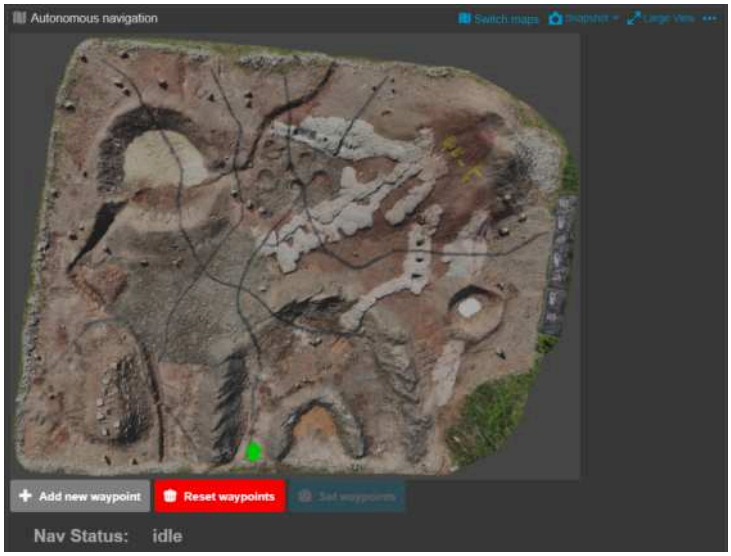



Figure 19: Navigation UI in OpenMCT in idle state

Additionally, Figure 20 shows the second state of this UI when the user toggles between the maps. In this view, the traditional map is replaced with a heightmap view of the same Mars yard, helping the operator to view the difficulty of the path and track the position in the $z$-axis using the UI.

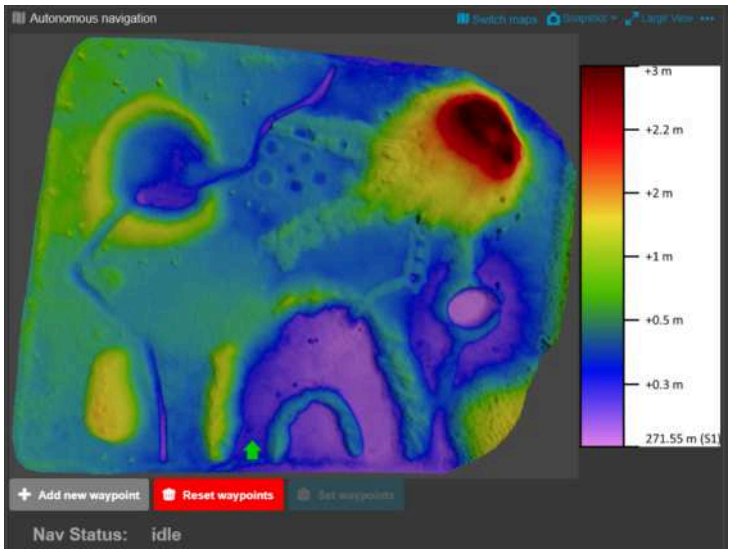



Figure 20: Navigation UI in OpenMCT in idle state with heightmap view

### 4.3.1 Path generation

To start creating a path, the operator clicks on “Add new waypoint” which opens the dialog shown in Figure 21.

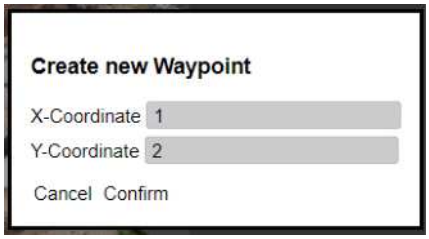



Figure 21: Waypoint-Creation Dialog in Navigation UI in OpenMCT

The user enters the $x$- and $y$-axis of the waypoint which will in turn set the waypoints on the map but not send anything to the `fhnw_waypoint_navigator`. Only upon clicking “Set waypoints”, an according message will be sent on the topic `/set_waypoints`. Figure 22 shows how the waypoints are visualized in the map view.




Figure 22: Waypoints visualization in Navigation UI in OpenMCT

Once the operator starts the navigation, the path is generated and visualized on the map as well as shown in Figure 23. Then, the rover will start driving towards the first waypoint. The operator can furthermore check the status of the navigation at all times below the buttons as seen in Figure 19.




Figure 23: Path visualization in Navigation UI in OpenMCT

## 4.4 Testing Days

At several Testing Days spread throughout the whole project period, different rover capabilities have been tested. The Testing Day in June provided the first possibility to test the autonomous navigation in its entirety. It was a great success as the rover did indeed visit the waypoints.



Figure 24: Rover driving autonomously at Testing Day

However, the Testing Day also provided insights into weakpoints. One was that the rover immediately starts driving and while driving adjusts its wheels. Due to that, it does not always drive in a very straight line as some curves might occur depending on the wheel state prior to driving. If two waypoints are very close to each other, this can result in the rover never reaching the designated waypoint. For longer distances the rover is able to correct the deviation and returns to the original path.

In an updated version, the navigation accounts for this and first sets the angles with no tangential velocity and only then sends the actual driving command. The threshold to detect if the change in angle requires this behavior is set to 2.6 rad or approximately 15 deg.

# 5 Final remarks, conclusion

This chapter summarizes the results of this project and discusses future improvements.

## 5.1 Pathfinding

Cost-efficient paths are generated visiting all given waypoints, including routing back to the starting point. The cost map generation features parameters enabling to tune the tradeoff between choosing the shortest route and avoiding steep terrain. A substantial improvement over the previous solution is the ability to mark areas as untraversable and requesting a new path with respect to the changed environment. This was not tested with the real rover yet though.

The pathfinding node can be improved with better error handling. Currently the node assumes it is correctly called with values on traversable terrain. Furthermore, some post-processing should be applied to the generated path, to allow smoother navigation with less turns.

Not all goals were met. Initially it was planned to have a smarter solution than an exhaustive search to find the order of waypoints resulting in the shortest path. Since the ERC did not increase the number of waypoints, however, this is not an issue. Reliable test results confirming the pathfinding stays within the timelimits proposed in the project declaration are also yet to be obtained. Another open task is to dynamically change the risk factor (`incline_alpha`).

## 5.2 Path traversal

The path traversal works and is a major improvement to the previous solution. It does already use some of the rovers abilities as demonstrated with the crab-steering but it could be further extended to use curves and arclines to drive along the calculated path.

The missing obstacle detection is an essential part and does in fact harm the performance of the rover's autonomous navigation. However, if the map provided by the organizers is detailed enough and does only miss very small obstacles such as fist-sized rocks, this should not pose a problem and the rover should be able to drive along the calculated path.

## 5.3 User Interface

The user interface is another major improvement to the previous implementation as the operators can now easily track the rover's movement and setting waypoints using the dialog is simple yet fulfills the requirements. Furthermore, the integration into OpenMCT ensures that the FHNW Rover Team only needs to use one control system for every aspect of the competition.

## 5.4 Outlook and final remarks

The whole autonomous navigation system remains to be tested in the real environment during ERC in September. Only then can final conclusions be drawn. However, both the tests in simulation as well as during the Testing Days were satisfactory.

There is still much potential for improvements in this newer navigation system. However, a very solid and robust foundation for future enhancements has been laid and the system will not have to be rebuilt again from scratch, but rather as an iterational improvement over the existing solution. The main goal of having a working and improved autonomous navigation system is still fulfilled.

# Bibliography

[1] "ROS - Robot Operating System." Accessed: Aug. 15, 2024. [Online]. Available: https://ros.org/

[2] "Meerkat - System76." Accessed: Aug. 15, 2024. [Online]. Available: https://system76.com/desktops/meerkat

[3] E. R. Challenge, "ERC 2024 Rules On-Site Formula Rev. 3.1." Accessed: Aug. 12, 2024. [Online]. Available: https://roverchallenge.eu/onsite-edition-files

[4] S. Covo and J. Smith, "European Rover Challenge - imvs06," 2022.

[5] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968, doi: 10.1109/TSSC.1968.300136.

[6] A. Stentz, "Optimal and efficient path planning for partially-known environments," in *Proceedings of the 1994 IEEE International Conference on Robotics and Automation*, 1994, pp. 3310–3317. doi: 10.1109/ROBOT.1994.351061.

[7] S. Koenig and M. Likhachev, "D*lite," in *AAAI/IAAI*, 2002. [Online]. Available: https://api.semanticscholar.org/CorpusID:208940224

[8] "FHNW Rover Techincal Documentation (restriced access)." Accessed: Aug. 15, 2024. [Online]. Available: https://fhnw-rover.pages.fhnw.ch/technical-documentation/organization/decision/dr1-cpp-nodes

[9] "NASA Open Mission Control Technologies - OpenMCT." Accessed: Aug. 16, 2024. [Online]. Available: https://software.nasa.gov/software/ARC-15256-1D

# Declaration of honesty

I/we hereby declare that any individual/pair/team- work submitted for assessment is entirely the product of my own/my own & my partner's/my own & my team's effort

- that I/we have correctly cited all text passages that do not originate from me, in accordance with standard academic citation rules (e.g. APA or IEEE), and that I have clearly mentioned all sources used;
- that I/we have declared in footnotes or in an "Index of auxiliary tools" all aids used (AI assistance systems such as chatbots [e.g., ChatGPT], translation [e.g., DeepL] paraphrasing [e.g., Quillbot]) or programming applications [e.g., Github Copilot] and indicated their use at the corresponding text passages;
- that I/we have acquired all intangible rights to any materials I may have used, such as images or graphics, or that these materials were created by me;
- that the topic, the thesis or parts of it have not been used in an assessment of another module, unless this has been expressly agreed with the lecturer in advance and is stated as such;
- that I am aware that my work may be checked for plagiarism and for third-party authorship of human or technical origin (artificial intelligence);
- that I am aware that the FHNW School of Engineering will pursue a violation of this declaration of authenticity and that disciplinary consequences (reprimand or expulsion from the study program) may result from this.

Windisch, August 16, 2024

**Name:** Miro Albrecht
**Signature:**

**Name:** Philippe Krüttli
**Signature:**

# Appendix

# A Original task declaration

Fachhochschule Nordwestschweiz

CG IP | ML Alg

## 24FS_IMVS01: FHNW Rover Navigation

| | | | | |
|---|---|---|---|---|
| **Betreuer:** | Christoph Stamm<br>Sandro Covo | | **Priorität 1** | **Priorität 2** |
| | | **Arbeitsumfang:** | P5 oder P6 | P5 oder P6 |
| | | **Teamgrösse:** | 2er Team | Einzelarbeit |
| **Sprachen:** | Deutsch oder Englisch | | | |
| **Studiengang:** | Informatik | | | |

**Ausgangslage**

Die European Rover Challenge (ERC) ist Europas grösste Weltraumrobotik-Challenge, welche seit 2014 jeweils im September in Polen (Kielce) stattfindet. An der Endausscheidung der Challenge 2023 haben in der Kategorie On-Site 25 Teams aus der ganzen Welt teilgenommen, um mit ihren selbstgebauten Robotern auf einer künstlichen Marslandschaft verschiedene Aufgaben der robotischen Raumfahrt zu simulieren. Mit dabei war auch ein interdisziplinäres Team der FHNW Technik. Es war die zweite Teilnahme der FHNW bei dieser Rover Challenge und der hervorragende 2. Schlussrang ist ein sehr positives Ergebnis, das für weitere Teilnahmen stark motiviert.
Die Rangierung eines Rovers hängt stark davon ab, wie viele und welche Aufgaben autonom vom Rover erledigt werden können. Eine zentrale Komponente, welche zukünftig autonom funktionieren soll ist die Navigation.



FHNW Rover beim ERC 2023 in Kielce (Polen)

**Ziel der Arbeit**

Ziel der Arbeit ist, dass der Rover autonom eine vorgegebene Liste von Wegpunkten im offenen Gelände erreichen kann. Dabei soll er einen möglichst sicheren und schnellen Weg wählen.

**Problemstellung**

Der zukünftige FHNW Rover wird einen neuen Drive-Train mit lenkbaren Rädern erhalten. Daher ist es notwendig, die Ansteuerung der Räder neu zu machen und sicherzustellen, dass der Rover gerade Strecken, Kurven und Rotationen präzise abfahren kann. Darüber hinaus soll der Rover in der Lage sein, ausgehend von einem 2.5-D Geländemodell und einer Liste von Wegpunkten, einen guten Weg von Wegpunkt zu Wegpunkt autonom zu finden und diesen abzufahren. Dabei müssen gefährliche Situationen wie etwa steile Hänge vermieden werden.

**Technologien/Fachliche Schwerpunkte/Referenzen**

- ROS2 und Nav2
- Pfadfindungsalgorithmen
- C++ oder Python
- Simulationen mit Gazebo

**Bemerkung**

Der FHNW-Rover basiert auf ROS2 und wird seit 2 Jahren entwickelt. Erfahrungen mit ROS2 oder ähnlichen Technologien sind von Vorteil.

# B Navigation task ERC

### 7.3.2 Navigation Task

***When all the planned Science tasks are finished the rover must find its way back to the MLMOD (Martian Lander Module) to deliver the collected samples that will be sent back to Earth. Based on the current location of the rover established with the data from satellite positioning system in the MMO (Medium Mars Orbit) the track back to the lander was uploaded to the rover's computer. Traverse must be performed in autonomous mode due to limited bandwidth and delays encountered while controlling the rover manually from the Earth.***

***During the traverse activities (in some of the locations on the way and due to difficult terrain) it is also required to use the drone that is part of the rover that has landed on the Mars. The main task of the drone is to fly over the surface of Mars in order to support reconnaissance activities – i.e. to detect potential obstacles impeding the movement of the rover, as well as to assess the value of the terrain in terms of future sampling activities. The autonomously moving drone will have to find and photograph two interesting structures so that the staff on Earth can make appropriate decisions. Due to the likelihood of a sandstorm, it should also be possible to temporarily land in a safe place and then continue the mission. The drone's destination will be MLMOD.***

#### *7.3.2.1 Traverse Sub-Task*

##### 7.3.2.1.1 General Description

This task is intended to demonstrate the system's ability for semi to fully autonomous traversal. The team shall develop a project which gradually evolves into a fully autonomous system, traversing and gathering important data on its way. At an early stage, the system can be decoupled with the operator in the loop, but all planning and parameter estimation must be done by the computer system itself. This limits the operator to navigate the rover blindly i.e., without access to visual or any other spatial information, however, any kind of data can be processed on-board, providing the operator with support information about the localisation and state of the system. A smart navigation strategy, sensor fusion and image data processing are essential in this task.

The teams will be given **30 minutes** to perform Navigation: Traverse Sub-Task. The Teams will be given 15 minutes before the Task start to prepare the rover for the traversal. This time is called the Preparation Phase. During most of the Preparation Phase, full access to the Rover is granted, the Rover is put in a dedicated area of the Mars Yard and the Teams are allowed to drive in the area, upload software, check the functionalities, etc.

##### 7.3.2.1.2 Task Scenario

a) Send the rover the start position and waypoint information
b) Reach the planned waypoints (4) according to the Navigation Plan
c) Return rover to its start position
d) After the traverse, present techniques used, visualise the system data, compare the results with the plan calculated at the beginning etc.

##### 7.3.2.1.3 General Requirements

a) Reach the 4 points defined by the judges in any chosen sequence (!)
b) Reach the defined point 5 (finish) as the last one (!)
c) Perform this task without video feedback in a fully autonomous mode in order to maximise the received scores
d) Perform this task with video feedback to receive 50% of points (100% possible for autonomy if video feed-based autonomy is implemented and presented to judges – each team is responsible for presenting proof of autonomy to judges)
e) GNSS navigation can be used (assumes existence of satellite constellation over Mars)! Scoring is reduced to 80% in this case (100% possible for autonomy if GNSS-based autonomy or combination of GNSS and

landmarks-based autonomy is implemented and presented to judges – each team is responsible for presenting proof of autonomy to judges)

f) Summarising the abovementioned scoring:

| Case Description | Traverse Points Possible | Autonomy Points Possible |
|---|---|---|
| 1. Video feed used<br>2. No autonomy implemented | 50% | 0% |
| 1. Video feed used<br>2. Autonomy implemented | 50% | 100% |
| 1. Video feed used<br>2. GNSS used<br>3. No autonomy implemented | 50% | 0% |
| 1. Video feed used<br>2. GNSS used<br>3. Autonomy implemented | 50% | 100% |
| 1. GNSS used<br>2. Video feed not used<br>3. No autonomy implemented | 80% | 0% |
| 1. GNSS used<br>2. Video feed not used<br>3. Autonomy implemented | 80% | 100% |
| 1. GNSS not used<br>2. Video feed not used<br>3. No autonomy implemented | 100% | 0% |
| 1. GNSS not used<br>2. Video feed not used<br>3. Autonomy implemented | 100% | 100% |

*Table 1: ERC 2024 Schedule*

1. It does not matter what method do you use for autonomy - either you have it (=100%) or you do not have it (=0%).
2. E.g. 'Video feed used' and 'GNSS used' means that you see video feed or current location of the rover on the screen IN THE CONTROL STATION and based on this you perform rover's maneuvers by the operator.
3. In case you do not use this information in the control station but the rover does it onboard in autonomous mode (=WITHOUT SENDING IT TO THE CONTROL STATION / OPERATOR) you will get 100% of points for autonomy (if it really works).

Please be aware that both video feed and GNSS will not make autonomy implementation easy. You will still have a lot of features (e.g. rocks, trenches, bumps, landmarks...) in the Mars Yard that will not be visible in the 3D model. So going directly from Location #1 to Location #2 (assuming e.g. GNSS only) still includes risk of the rover flipping over or being stuck.

g) ArUco markers placed on the MY can be used to navigate the rover

#### 7.3.2.1.4 Additional Information

a) Initial rover position and orientation will be drawn at the beginning of each trial from a set of designated locations and with a limited heading

b) The rover can be tele-operated, but only using the position and orientation estimate available. This data can be visualised in any form (e.g. projecting the rover's position on the arena map provided or a top view picture etc.)

c) If the rover needs to be moved for any reason, it can only be moved back to the last successfully reached waypoint and/or rotated towards any point or back to the starting point. In both cases, penalties for manual intervention apply

d) Technical Reports should include a list of all sensors, together with detailed information about working modes, ways they are used in the navigation task and how the rover will be operated. Teams are entitled to consult all solutions with the judges before submission. Documentation will be verified by the judges and in case of

any doubts, the Team may be asked to reconfigure devices and/or their control strategy. Any difference between the approved configuration and the one used during the challenge can lead to disqualification (0 points for this task)

e) Task arena:
   - Final map with grid coordinates and POIs (Point Of Interest) will be provided on the warm-up day, if not agreed otherwise
   - Two types of landmarks are foreseen: natural landmarks which are elements of the landscape placed on the map, e.g. craters, small embankments, hills and artificial landmarks, e.g. artificial points for localisation purposes. Artificial landmarks can contain characteristic hi-visibility labels, unique geometric figures, alphanumeric signs or an AR/QR tag matching a POI label on the map; An example landmark (A4 size) design using an ArUco marker is presented below:

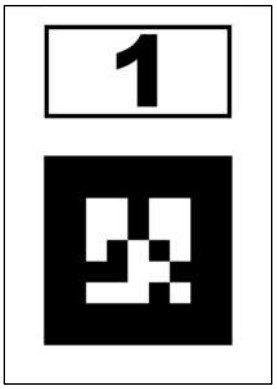



*Figure 3: Example of marker*

   - The artificial landmarks will be visible for cameras from different directions on a field (each landmark has 4 sides with identical faces) and will have a physical base that can be detected by proximity/range sensors (e.g. placed on an element of infrastructure or natural landmark)
   - At least two landmarks will be visible from the starting point
   - It must be taken into account that some of the landmarks may be obscured by terrain or other objects during a traverse
   - The team cannot place any additional passive landmarks or active beacons on the challenge field outside the starting area, but such elements can be deployed using the rover during the trial. All landmarks must be documented in the technical reports and presented for approval at least 10 working days before the submission of the final documentation. This equipment may be subject to negotiation, so teams should leave enough time to redesign/modify it in case of comments/rejection by the jury. Such equipment must comply with the other rules of the competition e.g. if active radio beacons are used, they must be compliant with the radio communication rules (see *Radio Communication* section) and described in RF form

f) The rover can be stopped and moved/rotated by team members when it is stuck or in case of any other technical problems. A judge shall be informed before any action is undertaken.

g) Details of the task, such as exact appearance of landmarks, location, map format, allowed custom landmarks and beacon types, etc. will be discussed with the teams and presented during the preliminary design phase in the rules. Teams are encouraged to initiate and actively participate in these discussions

h) Description of Landmarks – dimensions of landmarks are shown in the figure below.

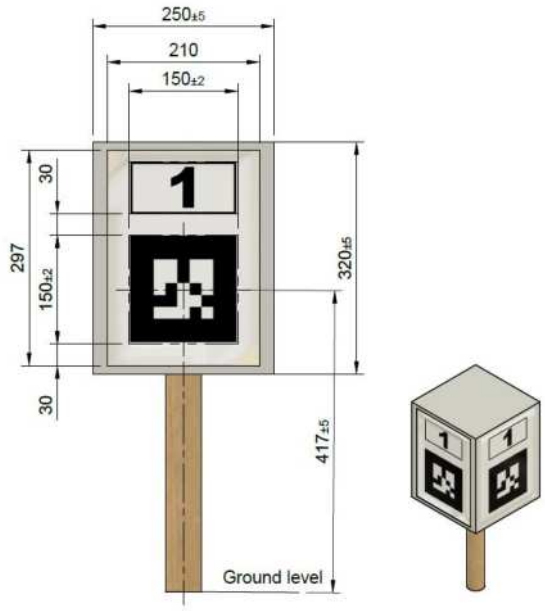



*Figure 4: Dimensions of landmarks*

Each artificial landmark face consists of:

- A white background of at least 210x297mm (A4 size)
- A landmark number indicator located in a 150x70mm (+-2mm) rectangle
- A marker square corresponding to the landmark number, with dimensions of 150x150mm (+-2mm) and at least 30mm white margin on each side, grid size of the tag is 5x5

Each artificial landmark pole consists of:

- A box with 4 landmark faces and dimensions of 250x250x310mm (+-5mm)
- A wooden pole that elevates the landmark box above the ground (according to the drawing)

Each landmark's coordinates are counted from an artificial point located in the landmark's pole axis – on a height of the landmark ARTag's centre point.

For the **list of ArUco markers** – please refer to the following generator.

The original ArUco markers (size of the marker 5x5, size of the black envelope 7x7) will be used (the final list to be confirmed in the *Update Report #3*). Markers ID No. 51-64 from the abovementioned generator are assumed at the moment.

*7.3.2.2 Droning Sub-Task*

#### 7.3.2.2.1 General Description

This task is intended to demonstrate the drone's ability for fully autonomous flight. The team shall develop an UAV or use the COTS one. Control system and autonomy of the drone shall be developed with the help of MATLAB/Simulink (software will be available for the teams). This task is governed by ONT (Oprogramowanie Naukowo Techniczne) – partner of the ERC and main distributor of Mathworks products in Poland. ONT plans to organize webinars for teams in order to introduce them to software and toolboxes that shall be used for this task. They will also serve as technical point of contact (mainly on the ERC's Slack).

#### 7.3.2.2.2 Task Scenario

a) Take off from the start location (Δ)
b) Take photo #1 (A)
c) Land in the emergency location (B) and take-off again
d) Take photo #2 (C)
e) Land in the finish location (Δ)

#### 7.3.2.3.3 General Requirements

a) Droning Sub-Task can be performed either autonomously (support from our partner will be given) or by remote control
b) GNSS navigation can be used (assumes existence of satellite constellation over Mars)!
c) Rover is not necessary for this task. The drone can be COTS or self-made. Self-made will get higher scores for technical excellence!
d) Each team has **15 minutes for preparation** to the task and **30 minutes for execution** of the task.
e) Teams must provide a Droning Sub-Task Report documenting the usage of MATLAB & Simulink and if applicable the control software and the drone design. The template can be found alongside templates of other reports.
f) The report shall be delivered using the following form: https://forms.gle/VXN7ZomEvH8mKCB68
g) Teams must submit photos taken by the drone at points A and C, one selected photo for each point, using the following form: https://forms.gle/S47ZbXCbkJtCT7N96
h) The photos must have one of the formats supported by the *imread* MATLAB function.

#### 7.3.2.2.4 Additional Information

a) Droning is to be performed in the drone cage (10x10x4m) that will be placed outside the MY.

b) Drone will follow the markers from the ArUco 5x5 family, incl. the black envelope (the same as in case of the Traverse task, see above). Start/end and Points A, B and C will use markers ID 101, 102, 103 and 104, in that order.

c) The assumed setup for this task is as follows:

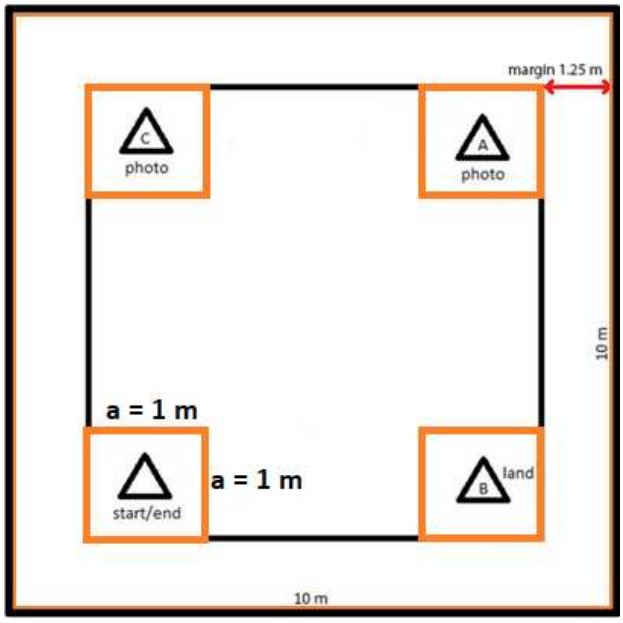



*Figure 5: The assumed cage for the Droning Sub-Task*

d) Around each point there will be a square with a side of 1 m marked on the ground with orange tape.

e) Same orange tape will be laid out along the edges of the cage.

f) The minimal required use of MATLAB/Simulink are *any* two documented uses of MATLAB/Simulink in the process of developing the drone or it's software.

g) The autonomy bonus points are awarded if the team developed and deployed the flight control logic used by the drone (not necessarily in MATLAB/Simulink).

h) Any calculations (e.g. image processing, sensor fusion, control logic) can be done "off-board" with just the results transmitted to the drone.

i) All ArUco markers on a landmark will be oriented in the same direction (and not, for example, all pointing towards the centre of the landmark).

j) Markers will be oriented in such a way, that the X/Y axes of the ArUco markers will be parallel/perpendicular to the edges of the cage.

k) Assuming Z axis pointing up, X axes of the markers will be pointing "right" (e.g. from start/end to B) and Y axes of the markers will be pointing "forward" (e.g. from start/end to C).

l) Description of Landmarks – dimensions of landmarks are shown in the figure below:

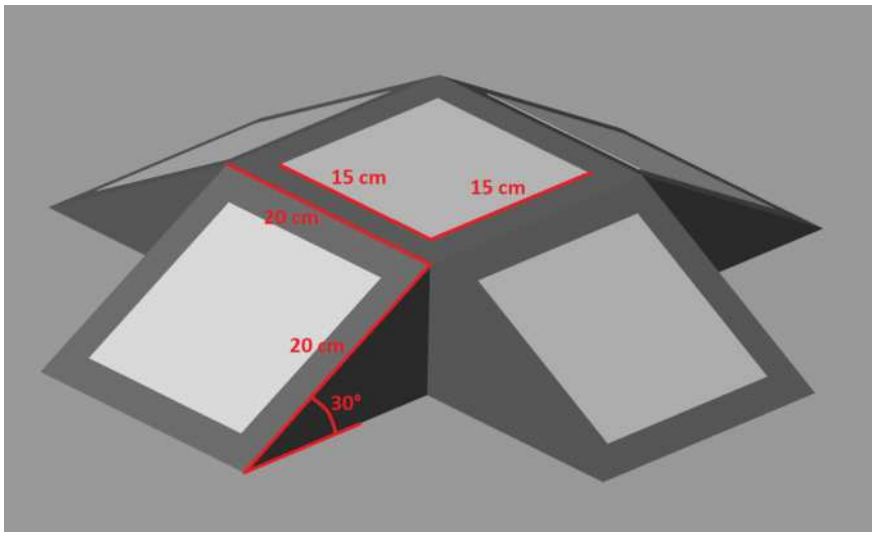



*Figure 5: Mock-up of a landmark used in Droning Sub-Task*

#### 7.3.2.2.4 Proper execution of task actions

a) Take-off:
- After take-off, the drone must be able to maintain a steady altitude for at least 3 seconds [eye test]

b) Reaching a point
- The drone must get roughly within the boundaries of the area around the landmark [eye test]

c) Taking photos
- The photos must contain at least one of the ArUco markers on the landmark and it must take up at least 5% of the image
- The image must be sufficiently sharp, that is, the ratio of length of the blur trail to the length of a side of a single square (bit) of the ArUco tag is no more than 1:10

d) Landing
- The centre point of the drone has to be within the area marked around the landmarks
- The drone has to turn off the rotors and wait for them to stop
- The drone is allowed to (fully or partially) land on top of the marker, as long as it doesn't damage the markers
- The drone can't get flipped or damaged during the landing for it to be valid

**NAVIGATION TASK – GENERAL COMMENTS:**

- **Traverse:** Remember about the required weighting and radio check before coming to the Navigation: Traverse starting point! Consider additional 20-30 minutes before as weight and radio check will have to be done in the teams' zone before arrival to the Mars Yard
- **Traverse & Droning:** Arrive early, you will have 15 minutes to prepare for the task (during this time you have to establish communication with the rover and place it on the start line)
- **Traverse & Droning:** Judge will explain you the task during the preparation phase
- **Traverse:** Please provide the planned traverse path to the judge before starting the task (in the Traverse Plan – template to be delivered in the *Update Report #3*)
- **Traverse & Droning:** You have 30 minutes to complete the task
- **Traverse:** If decided by the team, camera may be used for driving based on visual, however a significant decrease of points will be considered in this case in order to assure fair-play approach for the teams that have performed this task without vision or in autonomous mode. You can decide to use video at any time (e.g. in case you perform 1 of 4 points in autonomous mode you will still receive increased number of points for that one, scoring will be decreased only in case of the points reached with visual). Don't hesitate to make this decision in case of any problems – finally it is better to drive the rover than look how it stands on the Mars Yard
- **Traverse & Droning:** Part of the team in control of the rover or drone has to be in the tent with no view on the rover during the task (applicable to the part when no vision – see above – is used)
- **Traverse:** Only hardware changes/repairs performed during the task will be penalised, no penalties due to the safety reasons are expected – e.g. when rover is going to flip over on the crater's edge, stop/catch it and avoid damages. In case damage happened it is not possible to repair it without penalty.
- **Traverse & Droning:** Remember about preparing the strategy for each task. What to do in case of any problems? How long shall we wait or try to perform certain action before giving up and moving to the next one? Planning it in advance will save you some time